

Performance Insight API en Python

Migración desde ASP.NET, análisis de resultados de JMeter y estructura preparada para Vercel.

Autora: Cynthia

Fecha: 08/06/2026

Tecnologías: Python 3.12, FastAPI, JMeter 5.6.3, Vercel

Estado: Migración completada y validada en local

Objetivo del proyecto

El objetivo del proyecto es ofrecer una API que analice resultados de rendimiento, acepte muestras manuales o ficheros de JMeter y muestre un dashboard entendible con métricas técnicas y recomendaciones.

Migración realizada

La versión original estaba construida en ASP.NET. En esta migración se ha trasladado la API a Python usando FastAPI para facilitar un despliegue posterior en Vercel, manteniendo la idea funcional del proyecto.

- Se ha creado una entrada Vercel en `api/index.py`.
- La lógica se ha separado en modelos, rutas y servicios.
- Se ha mantenido el parser de JMeter y el dashboard HTML generado en servidor.
- Se ha preparado un `requirements.txt` y un `vercel.json` para despliegue.

Arquitectura de la solución

La aplicación queda organizada de forma modular. El fichero `api/index.py` expone la app a Vercel. `app/main.py` crea la instancia FastAPI, `app/routes/performance.py` publica los endpoints y `app/services` concentra el análisis, el parser de JMeter, la generación del dashboard y el almacenamiento del último informe.

Endpoints principales

Método	Ruta	Descripción
GET	/	Muestra el dashboard HTML generado por el servidor.

GET	/api/performance/health	Comprueba que la API está viva.
GET	/api/performance/current-report	Devuelve el último informe procesado.
POST	/api/performance/analyze	Analiza una carga manual enviada en JSON.
POST	/api/performance/dashboard	Calcula y guarda el dashboard desde muestras JSON.
POST	/api/performance/jmeter/upload	Sube y procesa un .jtl/.csv de JMeter.

Flujo de uso

```
python -m venv .venv
```

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

```
powershell -ExecutionPolicy Bypass -File .\scripts\run-fastapi.ps1
```

```
powershell -ExecutionPolicy Bypass -File .\scripts\run-jmeter-smoke.ps1  
-Domain dummyjson.com -Threads 1 -Loops 1
```

Resultados de las pruebas JMeter

Escenario	Target	Muestras	Media	P95	Máx	Errores	Lectura
Smoke local	GET /api/performance/health	6	1.33 ms	2 ms	2 ms	0%	Estado OK; la subida y el parser funcionan
Smoke público	dummyjson.com	4	555.5 ms	1640 ms	1640 ms	0%	Se detecta latencia alta en /products?d

Despliegue en Vercel

La estructura del proyecto queda preparada para desplegarse en Vercel gracias a FastAPI. El repositorio incorpora requirements.txt, .python-version, vercel.json y el entrypoint api/index.py. El siguiente paso operativo consiste en subir el repositorio a GitHub y conectarlo a una cuenta de Vercel.

- Vercel detecta Python y usa api/index.py como función.
- La API puede servirse sin necesidad de mantener ASP.NET.
- El estado del último informe está en memoria; para producción convendría persistencia externa.

Conclusiones

La migración a Python cumple el objetivo de acercar el proyecto a un despliegue compatible con Vercel. Además, se mantiene el valor académico principal de la práctica: construir una API propia, someterla a pruebas con JMeter y analizar los resultados mediante métricas y visualización.

Como mejora futura, sería recomendable desplegarla públicamente, añadir almacenamiento persistente para los informes y guardar histórico de ejecuciones para comparar campañas de rendimiento.